

**САНКТ-ПЕТЕРБУРГСКИЙ НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ
УНИВЕРСИТЕТ ИТМО**

Дисциплина: Бэк-энд разработка

Отчет

Лабораторная работа №2

Реализация микросервисов для сервиса бронирования ресторанов

Выполнил:

Ермаков Максим Олегович

Группа К3340

Проверил:

Добряков Д. И.

Санкт-Петербург

2026 г.

Задача

Опираясь на документ, сформированный в рамках домашней работы №4, необходимо реализовать разделение backend-приложения на микросервисы. Реализация должна подтвердить спроектированные границы сервисов, подход database-per-service и межсервисное взаимодействие через внутренние REST-вызовы.

Ход работы

Структура реализации

В папке labs/lab2 реализован набор Node.js/Express/TypeScript-сервисов. Каждый сервис запускается как отдельный процесс, имеет собственный порт и подключается только к своей базе данных PostgreSQL.

Сервис	Порт	Назначение
api-gateway	8100	Публичная точка входа /api/v1, авторизация, маршрутизация и композиция ответов
identity-service	8101	Пользователи, регистрация, вход, профиль и JWT introspection
catalog-service	8102	Рестораны, локации, кухни, фотографии и публикация ресторанов
menu-service	8103	Категории меню и позиции меню
reservation-service	8104	Столики, доступность столиков и бронирования
review-service	8105	Отзывы и пересчет рейтинга ресторанов

Разделение данных

Для доменных сервисов созданы отдельные базы данных: identity_db, catalog_db, menu_db, reservation_db и review_db. Docker Compose поднимает один контейнер PostgreSQL, а init-скрипт создает внутри него все необходимые логические базы данных.

Сервис	БД	Таблицы
identity-service	identity_db	users
catalog-service	catalog_db	locations, cuisines, restaurants, restaurant_cuisines, restaurant_photos
menu-service	menu_db	menu_categories, menu_items
reservation-service	reservation_db	restaurant_tables, reservations
review-service	review_db	reviews

Внешние связи между сервисами представлены идентификаторами, а не foreign key между БД. Например, menu-service хранит restaurant_id, но перед созданием категории меню проверяет ресторан через catalog-service.

API Gateway и авторизация

Клиент работает только с api-gateway. При обращении к защищенным endpoint-ам клиент передает JWT в заголовке Authorization. Gateway вызывает identity-service /internal/auth/introspect, проверяет токен и роль пользователя, после чего передает во внутренние сервисы заголовки X-User-Id, X-User-Role и X-Request-Id.

Внутренние сервисы доверяют пользовательскому контексту от gateway и используют его для проверки прав. Например, reservation-service разрешает обычному пользователю работать только со своими бронированиями, а административные операции требуют роль ADMIN.

Межсервисные сценарии

Сценарий	Реализация
Вход и регистрация	Gateway проксирует запросы в identity-service, который хранит пользователя и выпускает JWT
Поиск ресторанов	Gateway обращается к catalog-service и возвращает список опубликованных ресторанов
Детальная карточка ресторана	Gateway объединяет данные catalog, menu, reservation и review сервисов
Создание бронирования	reservation-service проверяет ресторан через catalog-service, пользователя через identity-service и отсутствие конфликта по времени
Создание отзыва	review-service проверяет ресторан, сохраняет отзыв, а затем обновляет рейтинг в catalog-service

Запуск и проверка

Для запуска используется PostgreSQL-контейнер и отдельные npm-скрипты сервисов. Типовая последовательность: npm install, docker compose up -d, npm run build, затем запуск start:identity, start:catalog, start:menu, start:reservation, start:review и start:gateway в отдельных терминалах.

Корректность реализации проверяется TypeScript-сборкой, health-check endpoint-ами каждого сервиса и smoke-тестом scripts/smoke-test.mjs. Smoke-тест последовательно выполняет вход администратора, создание справочников и ресторана, создание столика, меню и фото, публикацию ресторана, регистрацию пользователя, поиск ресторана, проверку доступности, создание бронирования и отзыв.

Проверка	Ожидаемый результат
npm run build	TypeScript-проект собирается без ошибок
GET /health каждого сервиса	Сервис возвращает статус ok
npm run smoke	Сценарий проходит до сообщения Smoke test passed
Просмотр БД через pgAdmin	В PostgreSQL видны отдельные базы данных сервисов

Вывод

В результате лабораторной работы монолитное backend-приложение было разделено на набор микросервисов согласно архитектуре из ДЗ4. Реализованы отдельные сервисы identity, catalog, menu, reservation и review, сохранен публичный API через gateway, а данные разделены по принципу database-per-service. Реализация подтверждает возможность дальнейшего развития проекта: контейнеризации каждого сервиса, добавления брокера сообщений и настройки CI/CD.